PROJECT AND TEAM INFORMATION

Project Title

Advanced Mathematical Expression Processing and Evaluation System

Team Name and Team Number

Team Name: The Postfix Crew **Team ID:** DSCPP-III-2026-T291

Team Members

S.No.	Name	Roll No.	Role	Email / Photo
1	Vanshika Prajapati	2510380242	Team Lead	 2510380242@geu.ac.in
2	Shagun	2510380250	Developer	 2510380250@geu.ac.in
3	Jai Bisht	2510011540	Testing	 2510011540@geu.ac.in

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

The project aims to develop an advanced mathematical expression processing and evaluation system using C++. The system allows user to input a mathematical expressions and performs validation, tokenization, conversion between different notations(prefix, infix, postfix), and evaluation of expressions. It also supports the generation and traversal of Expression Trees. The project is built using Data Structures and Object-Oriented Programming concepts such as Stack, Binary Tree, Recursion, Classes, Objects and Encapsulation. The system uses the Shunting-Yard algorithm for expression conversion and is developed using VS Code with Git and GitHub for collaborative development and version control.

Updated Project Approach and Architecture

System Workflow:

Input Expression →Validation →Tokenization →Conversion →Evaluation→Output

Major Components:

1. Input and Validation
2. Expression Processing and Conversion
3. Evaluation and Output as Expression Tree

The system is divided into five major modules: Input and Validation, Stack processing and Tokenization, Expression Conversion, Expression Evaluation, and Expression Tree. Stack-based processing is used for operator precedence and expression conversion. The Shunting-Yard approach is used for infix-to-postfix and infix-to-prefix conversion. Expression Trees are constructed and traversed to support expression processing. C++ classes, objects and encapsulation are used for the implementation.

Tasks Completed

Task / Contribution	Member	Status
System architecture, algorithm approach and module integration	Vanshika Prajapati	Completed
C++ implementation and expression-processing modules	Shagun	Completed
Testing, validation and implementation support	Jai Bisht	Completed / In Progress

Challenges / Roadblocks

1. Handling operator precedence correctly during expression processing.
2. Managing nested parentheses and different expression structures.
3. Integrating Stack processing with Expression Tree generation.

Solutions:

1. Stack-based precedence handling was used for systematic expression conversion.
2. Structured tree construction and traversal were used for Expression Tree processing.

Tasks Pending

Pending Task	Responsibility
Complete remaining implementation modules	Team
Complete system integration	Team
Conduct edge-case testing and validation	Jai Bisht / Team
Finalize documentation and demonstration	Team

Project Outcome / Deliverables

1. Working infix, postfix and prefix expression conversion and evaluation.
2. Stack-based expression processing.
3. Expression Tree generation and traversal.
4. Practical implementation of Data Structures and Object-Oriented Programming concepts.

Progress Overview

The project has progressed from the initial problem definition and system planning to module-level implementation and testing. The major system components and workflow have been structured, and the implementation focuses on expression validation, stack processing, conversion, evaluation and Expression Tree generation. Testing is being performed to validate simple expressions, precedence and parentheses handling, and Expression Tree processing.

Overall Progress: *50-60 percent*

Codebase Information

Development Platform: VS Code
Programming Language: C++
Version Control: Git and GitHub
Repository: https://github.com/vanshikaprjpt7/Advanced-Math-Expression-Engine
The project is written in C++ using object-oriented programming concepts. Separate Git branches are used for individual contributions . We have started bringing the completed parts of program together.
Individual Development Activity:

- Vanshika Prajapati: Worked on the program structure & organization of different modules.
- Shagun: Worked on implementing expression conversion & evaluation in C++.
- Jai Bisht: Helped run tests, check the program's output and identify errors.

Important Commits: *Add converter header for expression conversion 54bc57d*

Testing and Validation Status

Test Area	Purpose	Status
Input Validation/Tokenization	Checks for valid expressions, balanced parentheses, characters & correct tokens.	In Progress
Conversion & evaluation	Checks stack operations, Shunting-Yard Algorithm, & tests whether evaluation produces expected result.	In Progress
Expression Tree	Checks whether the Expression Tree is constructed & traversed correctly.	In Progress

Deliverables Progress

1. Input and Validation – Completed.
2. Stack Processing – In progress.
3. Expression Conversion – Completed.
4. Expression Evaluation – In progress.
5. Expression Tree & Tree traversal – In progress.

References

1. Shildt, H. – C++: The Complete Reference.
2. Balagurusamy, E. – OOP with C++.
3. Mark Allen Weiss – Data Structures & Algorithm Analysis in C++.
4. Git and GitHub documentation for version control and collaboration.